
AussieEcoLens — Melissa's Complete Deep Dive for Demo Preparation

---

What Melissa Owns and Why It Matters

Melissa's contributions are foundational infrastructure — DynamoDB, SNS, Cognito, and ECR. These are not supporting components. Every single Lambda function in the system depends on DynamoDB. Every upload triggers SNS. Every API call requires a valid Cognito token. ECR is what makes the inference Lambda possible. If any of these break, the entire system stops. The interviewer will almost certainly probe these areas deeply because they sit at the intersection of security, data storage, and messaging — all core cloud computing topics.

The Big Picture: How Melissa's Components Fit Together

Before going deep on each service, understand the relationships:

User registers/logs in → Cognito (auth)
→ JWT token issued
→ Every API call carries JWT → API Gateway validates against Cognito

File uploaded → Deduplication Lambda writes record → DynamoDB (storage)
→ Inference completes → Updates DynamoDB record with tags/status
→ SNS publishes notifications → Filtered to subscribers based on species

Inference Lambda container pulled from → ECR (container registry)

Architecture diagram → Shows all of the above visually

Melissa built/configured the services that sit underneath everything else.

Component 1: DynamoDB — Table Design and Schema

What is DynamoDB?

DynamoDB is AWS's managed NoSQL database. "Managed" means AWS handles server provisioning, replication, scaling, and backups — you never touch a database server. "NoSQL" means it does not use SQL, does not have a fixed schema (each item can have different attributes), and does not support joins across tables.

Why DynamoDB and not a relational database like RDS (PostgreSQL)?

This is the first question the interviewer will ask. There are three reasons:

First, Lambda functions scale horizontally. When many uploads happen simultaneously, AWS runs many Lambda instances in parallel. Relational databases use connection pools — each database connection is a resource that must be opened, maintained, and closed. If 100 Lambda instances all try to connect to an RDS instance simultaneously, you hit connection pool limits and get errors. DynamoDB uses an HTTP-based SDK — there are no persistent connections to manage. Each Lambda just makes an HTTP call to the DynamoDB endpoint. This scales to any number of concurrent Lambda invocations without connection pool issues.

Second, our data does not have a fixed schema. Each file record has a `tags` field that is a map of `{species: count}` pairs. Different files have different species detected. A relational database would require either a fixed columns approach (which breaks when you add new species) or a separate many-to-many tags table (which requires joins to query). DynamoDB's flexible document model stores the entire tags dict as a single attribute with no schema enforcement.

Third, DynamoDB is serverless — there are no servers to start, stop, or maintain. In AWS Academy, creating and managing RDS instances adds complexity around security groups, VPC configuration, and credential management. DynamoDB has none of that overhead.

The Table Configuration:

Table name: `AussieEcoLens`

Partition key: `file_url` (String type)

No sort key. This is a simple single-key table.

Billing mode: On-demand (pay per request). This is correct for a workload with unpredictable traffic. You don't need to provision read/write capacity units in advance.

Why `file_url` as the partition key?

The partition key is the primary lookup key. It must be unique per item. The `file_url` is unique per file — no two files have the same S3 URL because UUIDs are used in the key generation. When any Lambda needs to look up a specific file's record, it does so by `file_url`. DynamoDB uses the partition key to determine which physical partition (shard) the item is stored on — this is how it achieves O(1) lookups.

A partition key lookup (`get_item`) is the most efficient DynamoDB operation. It goes directly to the right partition. A `scan` reads every partition — much slower.

The Complete DynamoDB Item Structure:

When an item is first written by the deduplication Lambda, it looks like this:

```
{
  file_url:
"https://aussieecolens-media-276223878250.s3.amazonaws.com/uploads/<uuid>.jpg"
  checksum:  "d41d8cd98f00b204e9800998ecf8427e"
  bucket:    "aussieecolens-media-276223878250"
  key:       "uploads/<uuid>.jpg"
  status:    "processing"
}
```

After inference completes, the same item is updated with:

```
{
  file_url:       "https://...s3.amazonaws.com/uploads/<uuid>.jpg"   ← never changes
  checksum:       "d41d8cd98f00b204e9800998ecf8427e"
  bucket:         "aussieecolens-media-276223878250"
  key:            "uploads/<uuid>.jpg"
  status:         "complete"                                  ← updated
  tags:           {"cat": 3, "bird": 1}                       ← added
  file_type:      "image"                                     ← added
  thumbnail_url:  "https://...s3.amazonaws.com/thumbnails/cat/<uuid>.jpg"  ← added
  file_url_final: "https://...s3.amazonaws.com/media/cat/<uuid>.jpg" ← added
}
```

The `file_url` vs `file_url_final` split — why this exists:

After inference, the file is moved from `uploads/` to `media/<species>/`. The actual current location of the file changes. But the DynamoDB partition key (`file_url`) cannot be changed — DynamoDB items are identified by their partition key, and you can't rename the key without deleting and recreating the item. So `file_url` stays as the original `uploads/` URL forever. `file_url_final` holds the actual current S3 location.

Every Lambda that needs to access the actual file uses `item.get('file_url_final') or item['file_url']` — prefer the final URL, fall back to original if final doesn't exist. Every Lambda that deletes a DynamoDB record uses `Key={'file_url': item['file_url']}` — always the original key.

The `status` field and why it uses a reserved word workaround:

`status` is a DynamoDB reserved word. If you write `SET status = :s` in an `UpdateExpression`, DynamoDB throws a validation error. The fix is `ExpressionAttributeNames={'#st': 'status'}` in the `update_item` call, and then use `SET #st = :s` in the expression. This aliases the reserved word. Melissa configured the table, and this is a detail that comes up in the code — it's worth knowing why it exists.

DynamoDB Scan vs Query — a critical distinction:

A `query` uses the partition key (and optionally the sort key). It's fast — O(1). It goes straight to the right partition.

A `scan` reads every item in the table. It's slow — O(n). It costs read capacity proportional to the table size.

In our system, we use `scan` in multiple places because we can't query on non-partition-key attributes:

- Deduplication: scan for a matching `checksum` (not the partition key)
- Query Lambda: scan all `status = complete` records to filter by tags
- Delete/Tags: scan for matching `file_url_final` when a final URL is provided

The proper fix for production would be Global Secondary Indexes (GSIs) on `checksum`, `status`, and `file_url_final`. A GSI creates a second partition key on a different attribute, enabling O(1) lookups on that attribute. We didn't implement GSIs — this is a known limitation and a good "how would you improve this?" answer.

DynamoDB Pagination:

DynamoDB `scan` returns at most 1MB of data per call. If the table is larger, the response includes `LastEvaluatedKey`. Every scan in our code handles this:

```
response = table.scan(FilterExpression=...)
items = response.get('Items', [])
while 'LastEvaluatedKey' in response:
    response = table.scan(..., ExclusiveStartKey=response['LastEvaluatedKey'])
    items.extend(response.get('Items', []))
```

Without this, queries would silently miss records once the table exceeded 1MB.

Component 2: SNS — Topic, Filter Policies, and the Full Notifications Pipeline

What is SNS?

Amazon Simple Notification Service is a pub/sub (publish-subscribe) messaging service. A publisher sends a message to a topic. SNS delivers that message to all subscribers of the topic. Subscribers can be email addresses, SQS queues, Lambda functions, HTTP endpoints, and more. In our system, the publisher is the inference Lambda. The subscribers are user email addresses.

The SNS Topic:

Topic ARN:
`arn:aws:sns:us-east-1:276223878250:AussieEcoLens-Notifications`

Melissa created this topic. The ARN (Amazon Resource Name) is the globally unique identifier for any AWS resource. Every Lambda that interacts with SNS references this ARN from an environment variable (`SNS_TOPIC_ARN`).

The Filter Policy — this is the most important SNS concept:

Without filter policies, every subscriber would receive a notification every time any file is uploaded with any species. If a user only cares about wombats, they'd get spammed with cat, bird, and lizard notifications too. Filter policies solve this.

When a user subscribes, the Notifications Lambda attaches a filter policy to their subscription:

```
sns.subscribe(
    TopicArn=SNS_TOPIC_ARN,
    Protocol='email',
    Endpoint='user@example.com',
    Attributes={
        'FilterPolicy': json.dumps({'species': ['wombat', 'magpie']})
    }
)
```

This tells SNS: only deliver messages to this subscriber if the message has a `species` attribute whose value is in `['wombat', 'magpie']`.

When the inference Lambda publishes a message, it includes `MessageAttributes`:

```
sns.publish(
    TopicArn=SNS_TOPIC_ARN,
    Subject=f"AussieEcoLens: {tag} detected",
    Message=json.dumps({...}),
    MessageAttributes={
        'species': {'DataType': 'String', 'StringValue': tag}  # e.g. 'wombat'
    }
)
```

SNS compares the `species` attribute in the message against every subscription's filter policy. If the value matches, the message is delivered. If it doesn't match, that subscriber doesn't receive anything. This filtering happens inside SNS — the Lambda doesn't need to know who is subscribed to what.

One message is published per detected species tag. If an image contains 3 cats and 1 bird, two SNS messages are published — one with `species: cat`, one with `species: bird`. Each message is delivered only to subscribers who have `cat` or `bird` in their filter policy respectively.

The Subscription Confirmation Step:

After `sns.subscribe()` is called, SNS sends a confirmation email to the address. The email contains a "Confirm subscription" link. The user must click that link. Until they do, the subscription is in `PendingConfirmation` status and no messages are delivered to that address. After confirming, the subscription becomes `Confirmed` and messages start flowing.

This is a security mechanism — it prevents subscribing arbitrary email addresses without their consent.

The Unsubscribe Flow:

To unsubscribe, the code needs the subscription ARN (which is assigned by SNS when the subscription is confirmed). The Notifications Lambda can't just call `unsubscribe(email)` directly because SNS doesn't index subscriptions by email address. Instead, it calls `list_subscriptions_by_topic`, paginates through all subscriptions, finds the one where `Endpoint` matches the email, then calls `unsubscribe(SubscriptionArn=...)`.

Why SNS and not SQS?

SQS (Simple Queue Service) is a message queue. A message sits in the queue until a consumer pulls it. SQS doesn't support email delivery to end users. SNS is pub/sub — it pushes messages to subscribers immediately. The spec requires email notifications. SNS is the correct service for email push notifications.

Why SNS and not EventBridge?

EventBridge (formerly CloudWatch Events) is for routing events between AWS services. It supports filtering and routing to Lambda, SQS, SNS, and more. It's more complex and designed for service-to-service event routing, not end-user notifications. SNS with email subscriptions is the direct, correct tool for this use case. The spec even names SNS explicitly.

Melissa's Testing of the Notifications Pipeline:

The spec requires end-to-end testing of notifications. This means:

1. Subscribe an email to a specific species (e.g. "cat")

2. Confirm the subscription by clicking the link in the confirmation email
3. Upload an image that contains a cat
4. Wait for inference to complete (2-5 minutes on a cold start)
5. Verify that the SNS notification email arrives at the subscribed address
6. Verify that the email only arrives for the correct species (not for other species in the image)
7. Test filter policy correctness — if subscribed to "cat" only, uploading a wombat image should not trigger a notification

This is non-trivial to test because it requires the full pipeline to be running, and the latency between upload and notification can be several minutes due to inference cold start times.

Component 3: Cognito — User Pool Setup and Authentication Flow

What is Cognito?

Amazon Cognito is AWS's managed identity service. It handles user registration, email verification, login, password management, token issuance, and session management. The spec mandates Cognito for authentication — no alternative is permitted.

The User Pool:

User Pool ID: `us-east-1_A77QgwFr4` App Client ID: `kqn7o0djkqhbvel49mrob9elj`

Melissa created the User Pool. A User Pool is Cognito's user directory — it stores usernames, emails, password hashes, and account status. The App Client is the configuration that allows an application (the React frontend) to interact with the User Pool via the Cognito API.

What attributes were configured:

The spec requires: email, first name, last name, and password. In Cognito, `email` is a standard attribute. `given_name` and `family_name` are standard attributes for first and last name respectively. These were configured as required attributes on the User Pool. This means Cognito will not create an account without all four being provided.

Email was configured as the sign-in identifier (not a username). Users log in with their email address and password.

Email Verification:

Cognito automatically sends a verification email with a 6-digit code when a new account is created. The user must enter this code to confirm their account. Unconfirmed accounts cannot log in. This is configured in the User Pool settings — it's an Auto-Verified Attributes setting where `email` is in the list.

The JWT Token Flow — this is the core of how auth works:

When a user logs in successfully through the React frontend (using AWS Amplify's `signIn` function), Cognito returns three tokens:

1. **ID Token** — Contains user identity claims (email, given_name, sub, etc.). This is what our system uses. Signed with Cognito's RSA private key.
2. **Access Token** — Used for authorising Cognito API calls (e.g. getting user info). Not used directly in our API calls.
3. **Refresh Token** — Long-lived token used to get new ID/Access tokens after they expire without requiring re-login. Valid for 30 days by default.

The ID Token and Access Token expire after **1 hour** by default.

The frontend stores the ID token and sends it in every API call: `Authorization: Bearer <id_token>`.

The Cognito JWT Authoriser on API Gateway:

Melissa configured (or this was configured in coordination with the backend work) a Cognito User Pool Authoriser on API Gateway. This is what protects every API route.

When API Gateway receives a request, before forwarding it to any Lambda, it:

1. Extracts the `Authorization` header value (the Bearer token)
2. Checks the token signature against Cognito's public keys (fetched from the JWKS endpoint: `https://cognito-idp.us-east-1.amazonaws.com/us-east-1_A77QgwFr4/.well-known/jwks.json`)
3. Verifies the token hasn't expired
4. Verifies the token was issued for the correct audience (the App Client ID)
5. If all checks pass, forwards the request to the Lambda
6. If any check fails, returns a 401 Unauthorized immediately

This means no Lambda function ever needs to validate tokens itself. The validation is centralised at the API Gateway layer. This is the correct architectural pattern.

The GCP Proxy and Cognito:

The GCP Cloud Function sits between the frontend and API Gateway. The frontend sends the JWT to GCP. GCP forwards it unchanged in the `Authorization` header to API Gateway. GCP never decodes, validates, or stores the token — it's purely a pass-through. API Gateway is the one that validates the JWT against Cognito. This keeps all auth logic on AWS, which is where the spec requires it.

Why Cognito and not rolling your own auth?

Rolling your own authentication (storing passwords, issuing tokens, managing sessions) is error-prone, time-consuming, and a significant security risk. Cognito handles: password

hashing (bcrypt), email verification, account lockout, token expiry, key rotation, and compliance. The spec mandates it, but it's also the correct choice for a production cloud application.

AWS Academy Constraints on Cognito:

Under AWS Academy, `iam:CreateRole` is restricted. The `LabRole` is pre-created and shared. This means the Cognito User Pool cannot be configured with a custom Lambda trigger (e.g. a pre-sign-up Lambda that validates email domains) because that would require creating a new Lambda execution role. This was a constraint that was worked around by keeping Cognito configuration straightforward — standard sign-up, email verification, no custom triggers.

Component 4: ECR — Container Image Repository

What is ECR?

Amazon Elastic Container Registry is AWS's managed Docker container registry. It stores Docker images. Lambda can use container images from ECR as the deployment package instead of a zip file.

Repository URI:
`276223878250.dkr.ecr.us-east-1.amazonaws.com/aussieecolens-inference`

Melissa created this ECR repository. Without it, the inference Lambda container has nowhere to be pushed to and pulled from.

Why ECR and not Docker Hub?

Lambda can only pull container images from ECR in the same AWS account and region. It cannot pull from Docker Hub, GitHub Container Registry, or any public registry. ECR is the mandatory choice for Lambda container images.

The Container Image Pipeline:

1. On an EC2 instance (Linux x86_64), the Dockerfile is built with `docker build --platform linux/amd64 -t aussieecolens-inference .`
2. The image is tagged: `docker tag aussieecolens-inference:latest 276223878250.dkr.ecr.us-east-1.amazonaws.com/aussieecolens-inference:latest`
3. Docker is authenticated to ECR: `aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin 276223878250.dkr.ecr.us-east-1.amazonaws.com`

4. The image is pushed: `docker push 276223878250.dkr.ecr.us-east-1.amazonaws.com/aussieecolens-inference:latest`
5. The inference Lambda is configured with the image URI as its deployment package

Why EC2 for the build and not a local machine?

As covered in the backend deep-dive — Mac M-series uses ARM64. Lambda runs x86_64. Building on a Mac without `--platform linux/amd64` produces ARM64 binaries that fail on Lambda. EC2 runs Linux x86_64 natively, so binaries are compiled correctly without any cross-compilation flags. The EC2 instance (`i-054049cd1fd98d836`) must not be terminated — it's the build environment for future inference Lambda updates.

ECR Image Size:

The container image is large — PyTorch, MegaDetector, OpenCV, onnx2torch, plus their dependencies. This is why Lambda's 250MB zip limit makes a zip deployment impossible. ECR supports images up to 10GB. The actual image is several GB.

ECR and AWS Academy:

Under Academy, ECR repositories can be created and images can be pushed using the `LabRole` credentials. The Lambda execution role (`LabRole`) also has `ecr:GetAuthorizationToken` and `ecr:BatchGetImage` permissions, which are needed for Lambda to pull the image at invocation time.

Component 5: Architecture Diagram

Melissa led the construction of the architecture diagram based on the flow provided by Bharath. The diagram shows the full system from browser through GCP proxy to AWS API Gateway, all Lambda functions, S3 buckets, DynamoDB, SNS, ECR, and EC2.

Key things to know about the diagram for the demo:

The spec requires official architecture icons for all cloud providers. The diagram uses official AWS icons (from the AWS icon set) and official GCP icons (from the GCP Icons library in draw.io). The GCP Cloud Functions icon was embedded as an SVG data URI because draw.io's standard library doesn't include it natively.

The diagram reads left to right: User/Browser → GCP Cloud Function → API Gateway (with Cognito authoriser shown) → Lambda functions → downstream resources (S3, DynamoDB, SNS, ECR).

What the Interviewer Will Ask Melissa — Full Q&A Prep

"Why DynamoDB and not a relational database?"

Three reasons: Lambda concurrency causes connection pool exhaustion with RDS — DynamoDB uses stateless HTTP calls. Our tags data has a flexible schema unsuitable for fixed relational columns. DynamoDB is fully serverless with no connection or server management overhead.

"What is the partition key and why did you choose file_url?"

The partition key is the primary unique identifier. `file_url` is unique per file because UUIDs are used in key generation. It allows O(1) `get_item` lookups from any Lambda that knows the file URL.

"Why does the DynamoDB scan instead of query for most operations?"

A query only works on the partition key. Checking for duplicate checksums, filtering by status, and looking up by `file_url_final` all operate on non-partition-key attributes — these require scan. The production fix would be Global Secondary Indexes (GSIs) on those attributes.

"What is a GSI and how would it improve the system?"

A Global Secondary Index creates an alternate partition key on a different attribute. Adding a GSI on `checksum` would allow the deduplication Lambda to do an O(1) query instead of a full table scan. Adding a GSI on `status` would allow the query Lambda to retrieve only `complete` records without scanning everything.

"Explain how the SNS filter policy works end to end."

When a user subscribes, their subscription is created with a FilterPolicy: `{"species": ["cat"]}`. When inference publishes an SNS message with `MessageAttributes: {"species": "cat"}`, SNS evaluates the attribute against every subscription's filter policy. Only subscriptions whose filter policy includes `"cat"` receive the message. Subscriptions with different policies receive nothing.

"What happens if the user doesn't confirm the SNS subscription email?"

The subscription remains in `PendingConfirmation` state. SNS will not deliver any messages to that email address until the confirmation link is clicked. The subscription ARN returned is provisional and cannot be used for unsubscription until confirmed.

"What is a JWT and how does Cognito use it?"

JWT stands for JSON Web Token. It is a base64-encoded JSON payload containing claims (user ID, email, expiry time, issuer, audience) signed with Cognito's RSA private key. API Gateway validates the signature using Cognito's public keys fetched from the JWKS

endpoint. If the signature is valid and the token hasn't expired, the request is authorised. The signature prevents tampering — you cannot change the claims without invalidating the signature.

"What tokens does Cognito issue on login?"

Three tokens: ID Token (user identity claims, used in our Authorization header), Access Token (for Cognito API calls), and Refresh Token (used to get new ID/Access tokens after expiry, valid 30 days). ID and Access tokens expire after 1 hour.

"Why is the Cognito JWT passed through GCP without validation?"

GCP has no knowledge of Cognito. Validating a Cognito JWT on GCP would require GCP to fetch Cognito's JWKS endpoint, verify the RS256 signature, and check audience and expiry — essentially reimplementing Cognito auth on a different cloud. This adds complexity and duplication for no security gain, because API Gateway validates the token anyway. GCP acts purely as a transport layer.

"Why ECR and not Docker Hub for the inference Lambda image?"

Lambda can only pull container images from ECR in the same account and region. Docker Hub is not supported as a source for Lambda container images. ECR is the mandatory choice.

"Why was EC2 used to build the container and not a local machine?"

Lambda runs on Linux x86_64. Building Docker images on a Mac M-series (ARM64) produces ARM64 binaries that fail with Exec format error on Lambda's x86_64 runtime. EC2 runs Linux x86_64 natively so the build produces compatible x86_64 binaries without cross-compilation.

"What is the difference between SNS and SQS?"

SNS is pub/sub — it pushes messages to subscribers immediately. It supports email, SMS, Lambda, and HTTP subscribers. SQS is a queue — messages sit in the queue until a consumer pulls them. SQS does not support email delivery to users. For user-facing email notifications, SNS is the correct service.

"If a user uploads a file and nothing happens, how would you diagnose it?"

Check CloudWatch logs for the deduplication Lambda first — did it fire? If no logs, the S3 event trigger may not be configured. If the deduplication Lambda fired but inference never ran, check the async invocation — Lambda async failures go to a dead-letter queue if configured. Check the inference Lambda logs. If inference ran but DynamoDB wasn't updated, check the Lambda execution role permissions. If DynamoDB was updated but SNS notifications didn't arrive, check the subscription status (confirmed or pending), and check whether the filter policy matches the published message attributes.